

SECURITY LAB REPORT

# ARP Spoofing & Man-in-the-Middle Attack Lab

*Network traffic interception using ARP poisoning in a controlled VMware lab environment*

| Lab Type         | Platform                | Attack Vector        | Date         |
|------------------|-------------------------|----------------------|--------------|
| Network Security | VMware (Kali + Windows) | ARP Poisoning / MITM | May 29, 2026 |

# 1. Lab Overview

This lab demonstrates a classic Man-in-the-Middle (MITM) attack using ARP (Address Resolution Protocol) spoofing in an isolated virtual machine environment. By poisoning the ARP cache of both the victim and the gateway, a Kali Linux attacker machine was positioned between all network communications, enabling full traffic interception and analysis.

ARP spoofing exploits the stateless nature of the ARP protocol — since ARP has no authentication mechanism, any host on a local network can send gratuitous ARP replies claiming ownership of any IP address.

# 2. Lab Environment

## 2.1 Network Topology

All machines operated on VMware's NAT network — subnet 192.168.81.0/24:

| Machine        | OS         | IP Address     | MAC Address       | Role            |
|----------------|------------|----------------|-------------------|-----------------|
| Kali Linux     | Kali 2024  | 192.168.81.128 | 00:0c:29:79:7a:8e | Attacker / MITM |
| Windows 10     | Windows 10 | 192.168.81.129 | 00:0c:29:e9:39:95 | Victim          |
| VMware Gateway | —          | 192.168.81.2   | 00:50:56:f0:63:31 | Default Gateway |

## 2.2 Tools Used

- arpspoof (dsniff) — ARP packet crafting and spoofing
- Wireshark — Live packet capture and deep packet inspection
- tshark — CLI-based post-capture analysis
- ip neigh / arp — ARP cache inspection and cleanup

# 3. Attack Execution

## 3.1 Step 1 — Enable IP Forwarding

IP forwarding was enabled on Kali to transparently relay victim traffic to the real gateway, keeping the victim's internet connection alive throughout the attack:

```
echo 1 > /proc/sys/net/ipv4/ip_forward
```

Without this, Kali would drop all forwarded packets — the victim would lose connectivity and the attack would be detected immediately.

### 3.2 Step 2 — Bidirectional ARP Poisoning

Two simultaneous arpspoof processes were launched in separate terminals:

Terminal 1 — Poison Windows VM (tell victim Kali = gateway):

```
arpspoof -i eth0 -t 192.168.81.129 192.168.81.2
```

Terminal 2 — Poison Gateway (tell gateway Kali = victim):

```
arpspoof -i eth0 -t 192.168.81.2 192.168.81.129
```

### 3.3 Step 3 — Attack Verification

The Windows ARP cache was inspected before and after the attack:

| State  | Entry                  | MAC Address       | Verdict                |
|--------|------------------------|-------------------|------------------------|
| Before | 192.168.81.2 (gateway) | 00-50-56-f0-63-31 | Real MAC — Normal      |
| After  | 192.168.81.2 (gateway) | 00-0c-29-79-7a-8e | Kali's MAC — Poisoned! |

Windows now believed Kali was the default gateway. All outbound traffic was routed through the attacker machine before reaching the internet.

## 4. Traffic Capture & Analysis

### 4.1 Capture Summary

| Metric                    | Value                    |
|---------------------------|--------------------------|
| Total Packets Captured    | 280,525 frames           |
| Total Data Intercepted    | 348.7 MB                 |
| Capture Duration          | 368 seconds (~6 minutes) |
| Capture Interface         | eth0 (Kali Linux)        |
| ARP Spoofing Packets Sent | 404 ARP frames           |

4.2 Protocol Distribution

| Protocol    | Frames  | Data Volume | Notes                            |
|-------------|---------|-------------|----------------------------------|
| QUIC / UDP  | 239,260 | 284 MB      | YouTube video stream (encrypted) |
| TLS / HTTPS | 8,196   | 29.7 MB     | Encrypted web traffic            |
| TCP         | 40,130  | 64 MB       | General connections              |
| DNS         | 676     | 75 KB       | Domain lookups (plaintext!)      |
| ARP         | 404     | 17 KB       | Spoofing packets from Kali       |
| ICMP        | 27      | 4 KB        | Ping packets                     |

4.3 ARP Spoofing Evidence in Capture

Wireshark's duplicate IP-MAC detection flagged the poisoning in real time:

```
192.168.81.2 is at 00:0c:29:79:7a:8e
192.168.81.129 is at 00:0c:29:79:7a:8e [duplicate use of IP detected!]
```

4.4 Intercepted DNS Queries

Despite YouTube content being fully encrypted via QUIC/TLS, DNS queries revealed the victim's browsing activity in plaintext:

| Domain Queried             | Count | Service                      |
|----------------------------|-------|------------------------------|
| mozilla.cloudflare-dns.com | 228   | Firefox DNS-over-HTTPS       |
| youtube-ui.l.google.com    | 6     | YouTube interface            |
| www.youtube.com            | 4     | YouTube main site            |
| youtubei.googleapis.com    | 4     | YouTube API                  |
| www.google.com             | 6     | Google Search                |
| detectportal.firefox.com   | 6     | Firefox captive portal check |

## 5. Key Findings

- ARP spoofing successfully positioned Kali as a MITM — confirmed by poisoned ARP cache on victim and Wireshark duplicate-IP alerts.
- 280,525 packets (348 MB) were silently intercepted over 6 minutes with no disruption to victim connectivity.
- HTTPS/TLS and QUIC traffic remained encrypted — content was unreadable, demonstrating the effectiveness of transport-layer encryption.
- DNS queries exposed all visited domains in plaintext, revealing full browsing activity despite HTTPS content encryption.
- Wireshark automatically flagged ARP poisoning via duplicate IP-MAC binding detection.
- arpspoof (manual) outperformed Bettercap in this VMware NAT environment due to gateway MAC resolution issues.

## 6. Defenses & Mitigations

| Defense                      | How It Helps                                                            |
|------------------------------|-------------------------------------------------------------------------|
| Dynamic ARP Inspection (DAI) | Validates ARP packets against a DHCP snooping table on managed switches |
| Static ARP Entries           | Pins critical hosts (gateway) to their real MAC — cannot be overwritten |
| Encrypted DNS (DoH / DoT)    | Prevents DNS query interception even when MITM is active                |
| Network Monitoring / IDS     | Detects duplicate ARP entries and alerts on anomalous MAC changes       |
| VPN / TLS Everywhere         | Encrypts all application data so intercepted traffic remains unreadable |

## 7. Lab Cleanup

---

### 7.1 Kali Linux

```
# Stop arpspoof: Ctrl+C on both terminals
echo 0 > /proc/sys/net/ipv4/ip_forward
sudo ip neigh flush all
```

### 7.2 Windows VM (Administrator CMD)

```
arp -d *
```

Post-cleanup verification confirmed the Windows ARP cache restored the real gateway MAC (00-50-56-f0-63-31) and all connectivity was fully restored.

## 8. Conclusion

---

This lab successfully demonstrated a full ARP spoofing / MITM attack chain — from initial ARP cache poisoning through live traffic interception to forensic analysis of captured packets. The experiment confirms that ARP spoofing remains a highly effective attack on unprotected local networks.

The most significant finding is the metadata leakage through DNS: even with HTTPS encryption protecting content, an attacker positioned as MITM can build a complete picture of a victim's online activity through DNS queries alone. This underscores the importance of encrypted DNS protocols (DoH/DoT) as a complement to transport-layer encryption.

All activities were performed in a fully isolated VMware lab environment for educational purposes.